Parte IV

Planificación de procesos

Capítulo 24

Tipos de planificación

La *planificación de procesos* se refiere a cómo determina el sistema operativo al orden en que irá cediendo el uso del procesador a los procesos que lo vayan solicitando, y a las políticas que empleará para que el uso que den a dicho tiempo no sea excesivo respecto al uso esperado del sistema.

Hay tres tipos principales de planificación:

A largo plazo Decide qué procesos serán los siguientes en ser iniciados. Este tipo de planificación era el más frecuente en los sistemas de lotes (principalmente aquellos con *spool*) y multiprogramados en lotes; las decisiones eran tomadas considerando los requisitos pre-declarados de los procesos y los que el sistema tenía libres al terminar algún otro proceso. La planificación a largo plazo puede llevarse a cabo con periodicidad de una vez cada varios segundos, minutos e inclusive horas.

En los sistemas de uso interactivo, casi la totalidad de los que se usan hoy en día, este tipo de planificación no se efectúa, dado que es típicamente el usuario quien indica expresamente qué procesos iniciar.

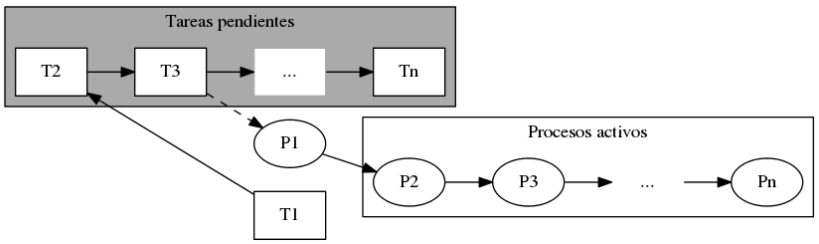


Figura 24.1: Planificador a largo plazo.

A mediano plazo Decide cuáles procesos es conveniente *bloquear* en determinado momento, sea por escasez/saturación de algún recurso (como la memoria primaria) o porque están realizando alguna solicitud que no puede satisfacerse momentáneamente; se encarga de tomar decisiones respecto a los procesos

conforme entran y salen del estado de *bloqueado* (esto es, típicamente, están a la espera de algún evento externo o de la finalización de transferencia de datos con algún dispositivo).

En algunos textos, al *planificador a mediano plazo* se le llama *agendador* (*scheduler*).

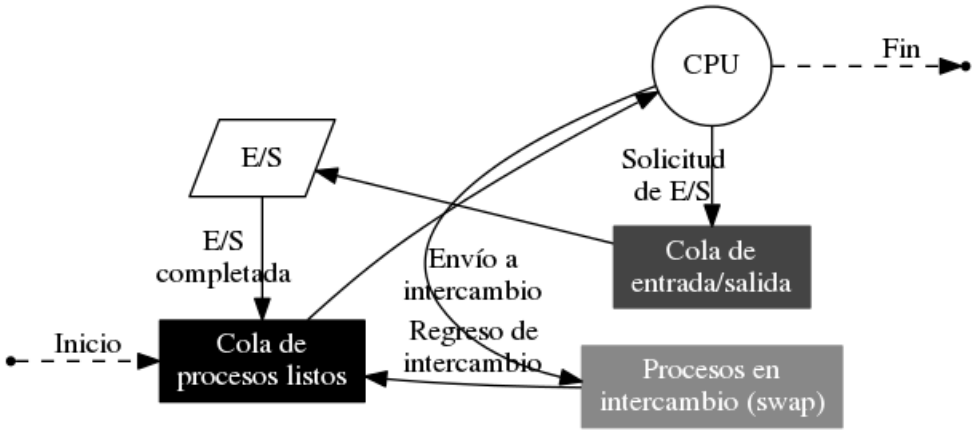


Figura 24.2: Planificador a mediano plazo, o *agendador*.

A corto plazo Decide cómo compartir *momento a momento* al equipo entre todos los procesos que requieren de sus recursos, especialmente el procesador. La planificación a corto plazo se lleva a cabo decenas de veces por segundo (razón por la cual debe ser código muy simple, eficiente y rápido); es el encargado de planificar *los procesos que están listos para ejecución*.

El *planificador a corto plazo* es también frecuentemente denominado *despachador* (*dispatcher*).

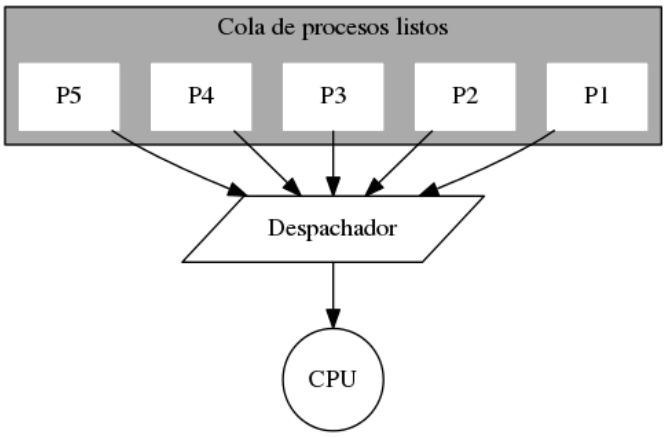


Figura 24.3: Planificador a corto plazo, o *despachador*.

Relacionando con los estados de un proceso abordados en la sección 19.1, y volviendo al diagrama entonces presentado (reproducido por comodidad de referencia en la figura 24.4), podrían ubicarse estos tres planificadores en las siguientes transiciones entre estados:

1. El planificador a largo plazo se encarga de *admitir* un nuevo proceso: la transición de *nuevo* a *listo*.
2. El planificador a mediano plazo maneja la *activación* y *bloqueo* de un proceso relacionado con *eventos*, esto es, las transiciones entre *en ejecución* y *bloqueado*, y entre *bloqueado* y *listo*.

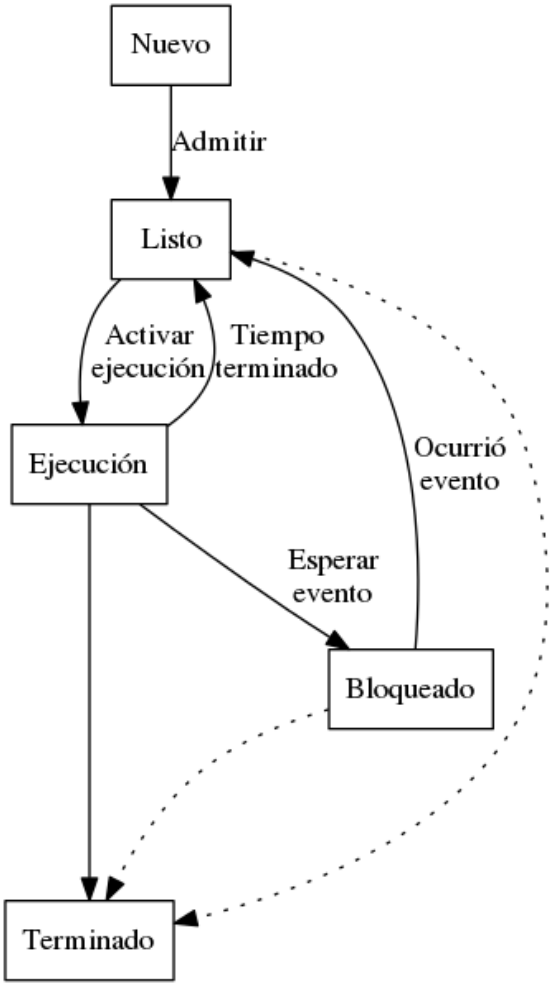


Figura 24.4: Diagrama de transición entre los estados de un proceso.

3. El planificador a corto plazo decide entre los procesos que están listos para ejecutarse y determina a cuál de ellos *activar*, y detiene a aquellos que *exceden su tiempo* de procesador —implementa las transiciones entre los estados *listo* y *en ejecución*.

En esta sección se trata particularmente el planificador *a corto plazo*, haciendo referencia como mucho a algunos efectos del planificador *a mediano plazo*.

24.1. Tipos de proceso

Como ya se ha visto, los procesos típicamente alternan entre *ráfagas* (periodos, en inglés *bursts*) en que realizan principalmente cómputo interno (están *limitados por* CPU, CPU-*bound*) y otras, en que la atención está puesta en transmitir los datos desde o hacia dispositivos externos (están *limitados por entrada-salida*, *I/O-bound*). Dado que cuando un proceso se suspende para realizar entrada-salida deja de estar *listo* (y pasa a estar *bloqueado*), y desaparece de la atención del planificador a corto plazo, en todo momento los procesos que están en ejecución y listos pueden separarse en:

Procesos largos Aquellos que *por mucho tiempo*[1] han estado en *listos* o en ejecución, esto es, procesos que estén

[1] ¿Cuánto es mucho? Dependerá de las políticas generales que se definan para el sistema.

en una larga ráfaga limitada por CPU.

Procesos cortos Los que, ya sea que en *este momento*[2] estén en una ráfaga limitada por entrada-salida y requieran atención meramente ocasional del procesador, o tienden a estar bloqueados esperando a eventos (como los procesos interactivos).

Por lo general se busca dar un tratamiento *preferente* a los procesos cortos, en particular a los interactivos. Cuando un usuario está interactuando con un proceso, si no tiene una respuesta *inmediata* a su interacción con el equipo (sea proporcionar comandos, recibir la respuesta a un *teclazo* o mover el puntero en el GUI) su percepción será la de una respuesta degradada.

24.2. Midiendo la respuesta

Resulta intuitivo que cada patrón de uso del sistema debe seguir políticas de planificación distintas. Por ejemplo, en el caso de un proceso interactivo, se buscará ubicar al proceso en una *cola* preferente (para obtener un tiempo de respuesta más ágil, para mejorar la percepción del usuario), pero en caso de sufrir demoras, es preferible buscar dar una respuesta *consistente*, aún si la respuesta *promedio* es más lenta. Esto es, si a todas las operaciones sigue una demora de un segundo, el usuario sentirá menos falta de control si en promedio tardan medio segundo, pero ocasionalmente hay picos de cinco.

Para este tema, en vez de emplear unidades temporales formales (p. ej. fracciones de segundo), es común emplear *ticks* y *quantums*. Esto es en buena medida porque, si bien en el campo del cómputo las velocidades de acceso y uso efectivo cambian constantemente, los conceptos y las definiciones permanecen. Además, al ser ambos parámetros ajustables, una misma implementación puede sobrevivir ajustándose a la evolución del hardware.

Tick Una fracción de tiempo durante la cual se puede realizar trabajo útil, esto es, usar el CPU sin interrupción[3]. El tiempo correspondiente a un tick está determinado por una señal (interrupción) periódica, emitida por el *temporizador* (timer). La frecuencia con que ocurre esta señal se establece al inicio del sistema. Por ejemplo, una frecuencia de *temporizador* de 100 Hertz implica que éste emitirá una señal cada 10 milisegundos. En Linux (a partir de la versión 2.6.8), un *tick* dura un milisegundo, en Windows, entre 10 y 15 milisegundos.

Quantum El tiempo mínimo que se permitirá a un proceso el uso del procesador. En Windows, dependiendo de la clase de proceso que se trate, un *quantum* durará entre 2 y 12 ticks (esto es, entre 20 y 180 ms), y en Linux, entre 10 y 200 ticks (10 y 200 milisegundos respectivamente).

¿Qué mecanismos o métricas se emplean para medir el comportamiento del sistema bajo determinado planificador? Partiendo de los siguientes conceptos, para un proceso p que requiere de un tiempo t de ejecución:

Tiempo de respuesta (T) Cuánto tiempo total es necesario para completar el trabajo pendiente de un proceso p, incluyendo el tiempo que está inactivo esperando ejecución (pero está en la cola de procesos listos).

Tiempo en espera ($E = T - t$) También referido como *tiempo perdido*. Del tiempo de respuesta total, cuánto tiempo p está listo y esperando ejecutar. Desde la óptica de p, se desearía que $E_p \to 0$

Proporción de penalización ($P = \frac{T}{t}$) Proporción del tiempo de respuesta en relación al tiempo de uso del procesador (en qué proporción fue penalizado el proceso).

[2] Y también, *este momento* debe ser interpretado con la granularidad acorde al sistema.

[3] Ignorando las interrupciones causadas por los dispositivos de entrada y salida y otras señales que llegan al CPU.

Proporción de respuesta ($R = \frac{t}{T}$) Inverso de P. Fracción del tiempo de respuesta durante la cual p pudo ejecutarse.

Para hacer referencia a un grupo de procesos con requisitos similares, todos ellos requiriendo de un mismo tiempo t, se emplea $T(t)$, $E(t) = T(t) - t$, $P(t) = \frac{T(t)}{t}$ y $R(t) = \frac{t}{T(t)}$.

Además de estos tiempos, expresados con relación al tiempo efectivo de los diversos procesos del sistema, es necesario considerar también:

Tiempo núcleo o *kernel* Tiempo que pasa el sistema en espacio de núcleo, incluyendo entre otras funciones[4] el empleado en decidir e implementar la política de planificación y los cambios de contexto. Este tiempo no se contabiliza cuando se calcula el tiempo del CPU utilizado por un proceso.

Tiempo de sistema Tiempo que pasa un proceso en espacio núcleo atendiendo el pedido de un proceso (syscall). Se incluye dentro del tiempo de uso del CPU de un proceso y suele discriminarse del tiempo de usuario.

Tiempo de usuario Tiempo que pasa un proceso en modo usuario, es decir, ejecutando las instrucciones que forman parte explícita y directamente del programa.

Tiempo de uso del procesador Tiempo durante el cual el procesador ejecutó instrucciones por cuenta de un proceso (sean en modo usuario o en modo núcleo).

Tiempo desocupado (*idle*) Tiempo en que la cola de procesos listos está vacía y no puede realizarse ningún trabajo.

Utilización del CPU Porcentaje del tiempo en que el CPU está realizando *trabajo útil*. Si bien conceptualmente puede ubicarse dicha utilización entre 0 y 100%, en sistemas reales se ha observado (Silberschatz, Galvin y Gagne 2010: 179) que se ubica en un rango entre 40 y el 90 por ciento.

Por ejemplo, si llegan a la cola de procesos listos:

Proceso	Ticks	Llegada
A	7	0
B	3	2
C	12	6
D	4	20

Si el tiempo que toma al sistema efectuar un cambio de contexto es de un *tick*, y la duración de cada *quantum* es de cinco ticks, en un ordenamiento de ronda,[5] se observaría un resultado como el que ilustra la figura 24.5.

Al considerar al tiempo ocupado por el núcleo como un proceso más, cuyo trabajo en este espacio de tiempo finalizó junto con los demás,[6] se obtiene por resultado:

[4] Estas funciones incluyen principalmente la atención a interrupciones, el servicio a llamadas al sistema, y cubrir diversas tareas administrativas.

[5] Este mecanismo se presentará en breve, en la sección 25.3.

[6] Normalmente *no* se considera al núcleo al hacer este cálculo, dado que en este ámbito todo el trabajo que hace puede verse como *burocracia* ante los resultados deseados del sistema.

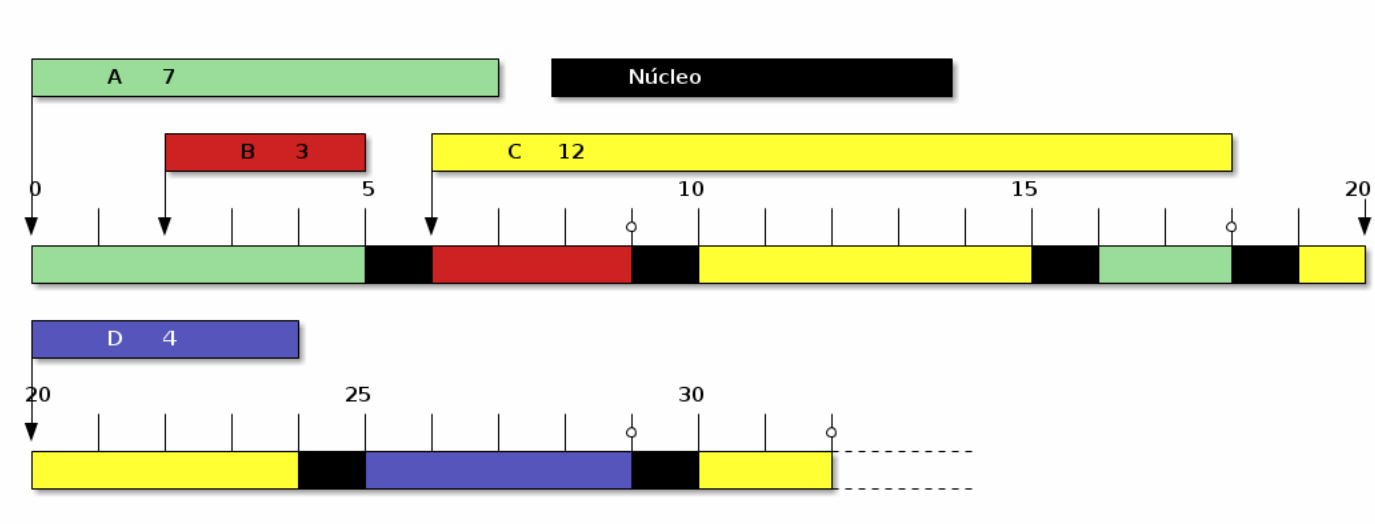


Figura 24.5: Ejecución de cuatro procesos con *quantums* de cinco *ticks* y cambios de contexto de dos *ticks*.

Proceso	t	T	E	P	R
A	7	18	11	2.57	0.389
B	3	7	4	2.33	0.429
C	12	26	14	2.17	0.462
D	4	9	5	2.25	0.444
Promedio *útil*	6.5	15	8.50	2.31	0.433
Núcleo	6	32	26	5.33	0.188
Promedio total	6.4	18.4	12.00	2.88	0.348

Abordando cada proceso, para obtener T se parte del momento en que el proceso llegó a la cola, no el punto de inicio de la línea de tiempo. En este caso, dado que el núcleo *siempre* está en ejecución, se asume que inició también en 0.

Respecto al patrón de llegada y salida de los procesos, se lo maneja también basado en una relación: partiendo de una *frecuencia α de llegada* promedio de nuevos procesos a la cola de procesos listos, y el *tiempo de servicio requerido* promedio β, se define el *valor de saturación* ρ como $\rho = \frac{\alpha}{\beta}$.

Cuando $\rho = 0$, nunca llegan nuevos procesos, por lo cual el sistema estará eventualmente *desocupado*. Cuando $\rho = 1$, los procesos son despachados al mismo ritmo al que van llegando. Cuando $\rho > 1$, el ritmo de llegada de procesos es mayor que la velocidad a la cual la computadora puede darles servicio, con lo cual la cola de procesos listos tenderá a crecer (y la calidad de servicio, la proporción de respuesta R, para cada proceso se decrementará).

Capítulo 25

Algoritmos de planificación

El planificador a corto plazo puede ser invocado cuando un proceso se encuentra en algunas de las cuatro siguientes circunstancias:

1. Pasa de estar *ejecutando* a estar *en espera* (por ejemplo, por solicitar una operación de E/S, esperar a la sincronización con otro proceso, etcétera).
2. Pasa de estar *ejecutando* a estar *listo* (por ejemplo, al ocurrir la interrupción del temporizador, o de algún evento externo).
3. Deja de estar *en espera* a estar *listo* (por ejemplo, al finalizar la operación de E/S que solicitó).
4. Finaliza su ejecución, y pasa de *ejecutando* a *terminado*.

En el primer y cuarto casos, el sistema operativo siempre tomará el control;[1] un sistema que opera bajo *multitarea apropiativa* implementará también el segundo y tercer casos, mientras que uno que opera bajo *multitarea cooperativa* no necesariamente reconocerá dichos estados.

Ahora, para los algoritmos a continuación, cabe recordar que se trata únicamente del *despachador*. Un proceso siempre abandonará la cola de procesos listos al requerir de un servicio del sistema.

Para todos los ejemplos que siguen, los tiempos están dados en *ticks*; no es relevante a cuánto *tiempo de reloj* éstos equivalen, sino el rendimiento relativo del sistema entero ante una carga dada.

La presente sección está basada fuertemente en el capítulo 2 de *An operating systems vade mecum* (Finkel 1988).

25.1. Objetivos de la planificación

Los algoritmos que serán presentados a continuación son respuestas que intentan, de diferentes maneras y desde distintos supuestos base, darse a los siguientes objetivos principales (tomando en cuenta que algunos de estos objetivos pueden ser mutuamente contradictorios):

Ser justo Debe tratarse de *igual manera* a todos los procesos que compartan las mismas características,[2] y nunca postergar indefinidamente uno de ellos.

Maximizar el rendimiento Dar servicio a la mayor parte de procesos por unidad de tiempo.

Ser predecible Un mismo trabajo debe tomar aproximadamente la misma cantidad de tiempo en completarse independientemente de la carga del sistema.

[1] En el primer caso, el proceso entrará en el dominio del *planificador a mediano plazo*, mientras que en el cuarto saldrá definitivamente de la lista de ejecución.

[2] Un algoritmo de planificación puede *priorizar* de diferente manera a los procesos según distintos criterios, sin por ello dejar de ser justo, siempre que dé la misma prioridad y respuesta a procesos equivalentes.

Minimizar la sobrecarga El tiempo que el algoritmo pierda en *burocracia* debe mantenerse al mínimo, dado que éste es tiempo de procesamiento útil perdido.

Equilibrar el uso de recursos Favorecer a los procesos que empleen recursos subutilizados, penalizar a los que peleen por un recurso sobreutilizado causando contención en el sistema.

Evitar la postergación indefinida Aumentar la prioridad de los procesos más *viejos*, para favorecer que alcancen a obtener algún recurso por el cual estén esperando.

Favorecer el *uso esperado* del sistema En un sistema con usuarios interactivos, maximizar la prioridad de los procesos que sirvan a solicitudes iniciadas por éste (aun a cambio de penalizar a los procesos *de sistema*)

Dar preferencia a los procesos que *podrían causar bloqueo* Si un proceso de baja prioridad está empleando un recurso del sistema por el cual más procesos están esperando, favorecer que éste termine de emplearlo más rápido

Favorecer los procesos con un *comportamiento deseable* Si un proceso causa muchas demoras (por ejemplo, atraviesa una ráfaga de entrada/salida que le requiere hacer muchas llamadas a sistema o interrupciones), se le puede penalizar porque degrada el rendimiento global del sistema

Degradarse suavemente Si bien el nivel ideal de utilización del procesador es al 100%, es imposible mantenerse siempre a este nivel. Un algoritmo puede buscar responder con la menor penalización a los procesos preexistentes al momento de exceder este umbral.

25.2. Primero llegado, primero servido (FCFS)

El esquema más simple de planificación es el *primero llegado, primero servido* (*first come, first serve*, FCFS). Este es un mecanismo cooperativo, con la mínima lógica posible: cada proceso se ejecuta en el orden en que fue llegando, y hasta que *suelta el control*. El despachador es muy simple, básicamente una cola FIFO.

Para comparar los distintos algoritmos de planificación que serán presentados, se dará el resultado de cada uno de ellos sobre el siguiente juego de procesos (Finkel 1988: 35):

Proceso	Tiempo de llegada	t	Inicio	Fin	T	E	P
A	0	3	0	3	3	0	1
B	1	5	3	8	7	2	1.4
C	3	2	8	10	7	5	3.5
D	9	5	10	15	6	1	1.2
E	12	5	15	20	8	3	1.6
Promedio		4			6.2	2.2	1.74

Si bien un esquema FCFS reduce al mínimo la *sobrecarga administrativa* (que incluye, tanto al tiempo requerido por el planificador para seleccionar al siguiente proceso, como el tiempo requerido para el cambio de contexto), el rendimiento percibido por los últimos procesos en llegar (o por procesos cortos llegados en un momento inconveniente) resulta inaceptable.

Este algoritmo dará servicio y salida a todos los procesos siempre que $\rho \leq 1$. En caso de que se sostenga $\rho > 1$, la demora para iniciar la atención de un proceso crecerá cada vez más, cayendo en una cada vez mayor inanición.

Puede observarse que FCFS tiene características claramente inadecuadas para trabajo interactivo, sin embargo, al no requerir de hardware de apoyo (como un temporizador) sigue siendo ampliamente empleado.

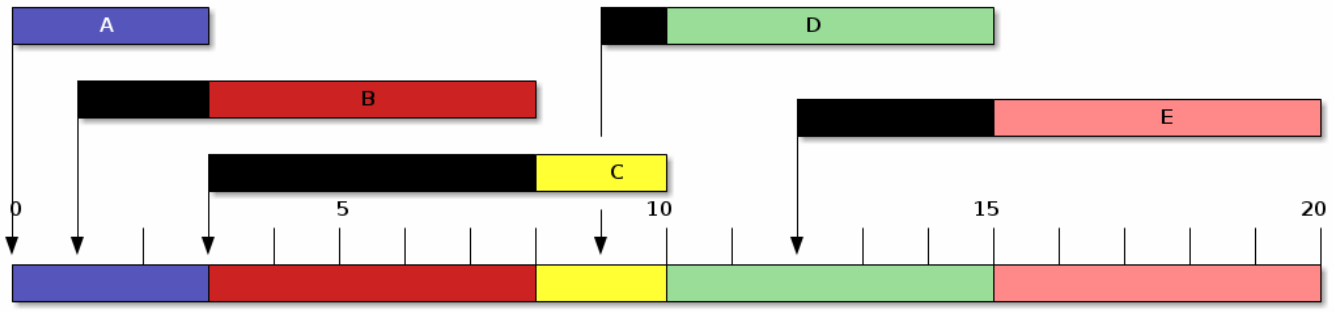


Figura 25.1: Primero llegado, primero servido (FCFS).

25.3. Ronda (*Round Robin*)

El esquema *ronda* busca dar una relación de respuesta buena, tanto para procesos largos como para los cortos. La principal diferencia entre la ronda y FCFS es que en este caso sí emplea multitarea apropiativa: cada proceso que esté en la lista de procesos listos puede ejecutarse por un sólo *quantum* (q). Si un proceso no ha terminado de ejecutar al final de su *quantum*, será interrumpido y puesto al final de la lista de procesos listos, para que espere a su turno nuevamente. Los procesos que sean *despertados* por los planificadores a mediano o largo plazos se agregarán también al final de esta lista.

Con la misma tabla de procesos presentada en el caso anterior (y, por ahora, ignorando la sobrecarga administrativa provocada por los cambios de contexto) se obtienen los siguientes resultados:

Proceso	Tiempo de llegada	t	Inicio	Fin	T	E	P
A	0	3	0	6	6	3	2.0
B	1	5	1	11	10	5	2.0
C	3	2	4	8	5	3	2.5
D	9	5	9	18	9	4	1.8
E	12	5	12	20	8	3	1.6
Promedio		4			7.6	3.6	1.98

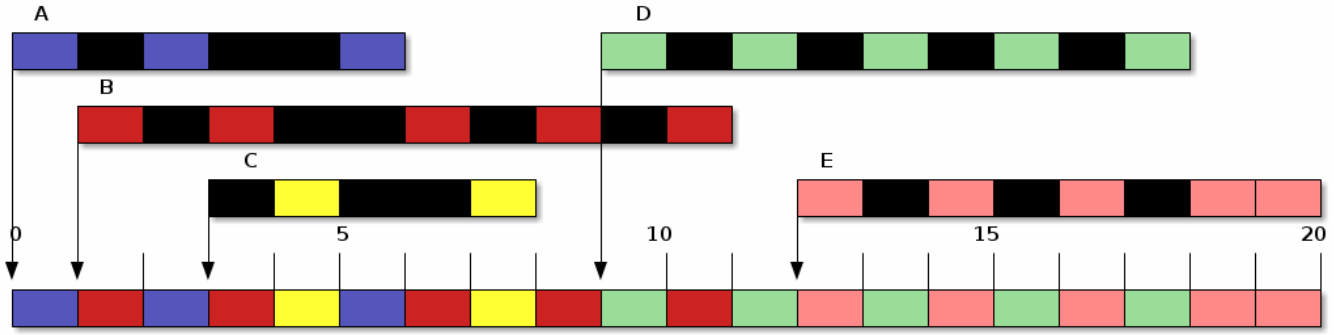


Figura 25.2: Ronda (*Round Robin*).

La *ronda* puede ser ajustada modificando la duración de q. Conforme se incrementa q, la ronda tiende a convertirse en FCFS —si cada *quantum* es arbitrariamente grande, todo proceso terminará su ejecución dentro

de su *quantum*; por otro lado, conforme decrece q, se tiene una mayor frecuencia de cambios de contexto; esto llevaría a una mayor ilusión de tener un procesador dedicado por parte de cada uno de los procesos, dado que cada proceso sería incapaz de notar las *ráfagas* de atención que éste le da (avance rápido durante un periodo corto seguido de uno sin avance). Claro está, el procesador simulado sería cada vez más lento, dada la fuerte penalización que iría agregando la sobrecarga administrativa.

Finkel 1988: 35 se refiere a esto como el *principio de la histéresis*: *hay que resistirse al cambio*. Como ya se mencionó, el algoritmo FCFS mantiene al mínimo posible la sobrecarga administrativa, y –aunque sea marginalmente– resulta en mejor rendimiento global.

Si se repite el análisis anterior bajo este mismo mecanismo, pero con un *quantum* de cuatro *ticks*, el resultado es:

Proceso	Tiempo de llegada	t	Inicio	Fin	T	E	P
A	0	3	0	3	3	0	1.0
B	1	5	3	10	9	4	1.8
C	3	2	7	9	6	4	3.0
D	9	5	10	19	10	5	2.0
E	12	5	14	20	8	3	1.6
Promedio		4			7.2	3.2	1.88

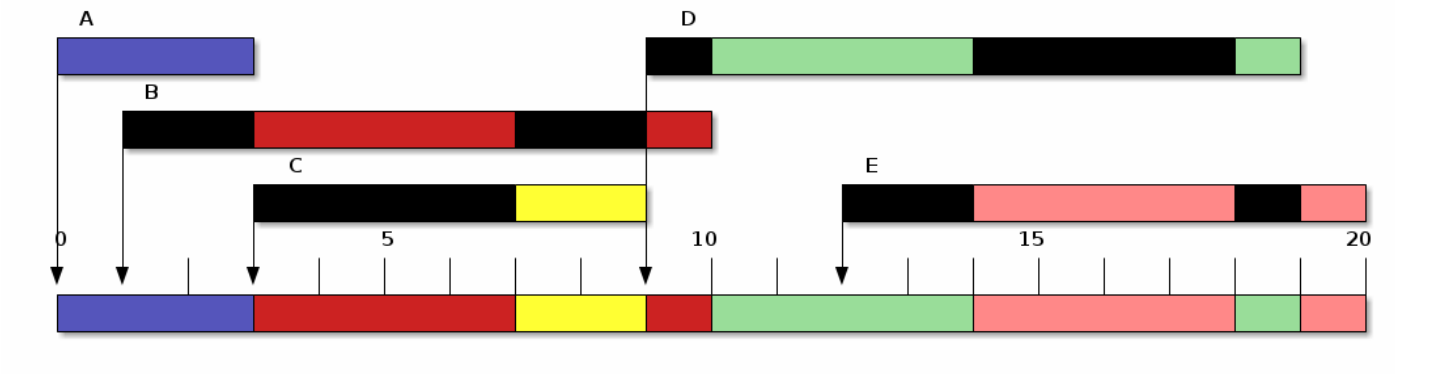


Figura 25.3: Ronda (*Round Robin*), con $q = 4$.

Si bien aumentar el *quantum* mejora los tiempos promedio de respuesta, aumentarlo hasta convertirlo en un FCFS efectivo degenera en una penalización a los procesos cortos, y puede llevar a la inanición cuando $\rho > 1$. Mediciones estadístcias apuntan a que típicamente el *quantum* debe mantenerse inferior a la duración promedio del 80% de los procesos (Silberschatz, Galvin y Gagne 2010: 188).

25.4. El proceso más corto a continuación (SPN, *shortest process next*)

Cuando no se tiene la posibilidad de implementar multitarea apropiativa, pero se requiere de un algoritmo más *justo*, contando con información *por anticipado* acerca del tiempo que requieren los procesos que forman la lista, puede elegirse el más corto de los presentes.

Ahora bien, es muy difícil contar con esta información antes de ejecutar el proceso. Es más frecuente buscar *caracterizar* las necesidades del proceso: ver si durante su historia de ejecución[3] ha sido un proceso tendiente a manejar ráfagas *limitadas por entrada-salida* o *limitadas por procesador*, y cuál es su tendencia actual.

[3]Cabe recordar que todos estos mecanismos se aplican al *planificador a corto plazo*. Cuando un proceso se bloquea esperando una

Para estimar el tiempo que requerirá un proceso p en su próxima invocación, es común emplear el *promedio exponencial* e_p. Se define un *factor atenuante* $0 \leq f \leq 1$, que determinará qué tan reactivo será el promedio obtenido a la última duración; es común que este valor sea cercano a 0.9.

Si el proceso p empleó q *quantums* durante su última invocación,

$$e'_p = fe_p + (1 - f)q$$

Se puede tomar como *semilla* para el e_p inicial un número elegido arbitrariamente, o uno que ilustre el comportamiento actual del sistema (como el promedio del e_p de los procesos actualmente en ejecución). La figura 25.5 presenta la predicción de tiempo requerido que determinado proceso va obteniendo en sus diversas entradas a la cola de ejecución, basado en su comportamiento previo, con distintos factores atenuantes.

Empleando el mismo juego de datos de procesos que se ha venido manejando como resultados de las estimaciones, se obtiene el siguiente resultado:

Proceso	Tiempo de llegada	t	Inicio	Fin	T	E	P
A	0	3	0	3	3	0	1.0
B	1	5	5	10	9	4	1.8
C	3	2	3	5	2	0	1.0
D	9	5	10	15	6	1	1.2
E	12	5	15	20	8	3	1.6
Promedio		4			5.6	1.6	1.32

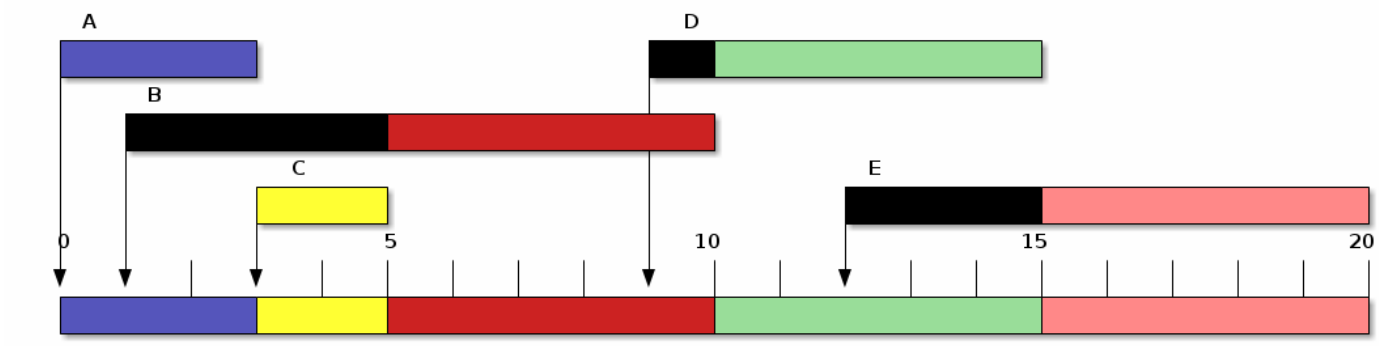


Figura 25.4: El proceso más corto a continuación (SPN).

Como era de esperarse, SPN favorece a los procesos cortos. Sin embargo, un proceso largo puede esperar mucho tiempo antes de ser atendido, especialmente con valores de ρ cercanos o superiores a 1 — un proceso más largo que el promedio está predispuesto a sufrir inanición.

En un sistema poco ocupado, en que la cola de procesos listos es corta, SPN generará resultados muy similares a los de FCFS. Sin embargo, puede observarse en el ejemplo que con sólo una permutación en los cinco procesos ejemplos (B y C), los factores de penalización a los procesos ejemplo resultaron muy beneficiados.

operación de E/S, sigue en ejecución, y la información de contabilidad del mismo sigue alimentándose. SPN se "nutre" precisamente de dicha información de contabilidad.

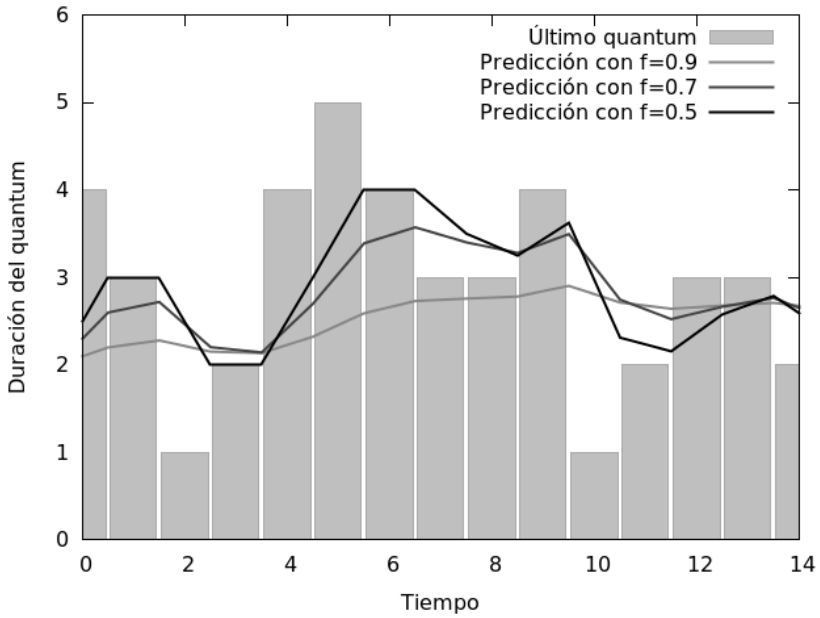


Figura 25.5: Promedio exponencial (predicción de próxima solicitud de tiempo) de un proceso.

25.4.1. SPN apropiativo (PSPN, *preemptive shortest process next*)

Finkel (1988: 41) apunta que, a pesar de que intuitivamente daría una mayor ganancia combinar las estrategias de SPN con un esquema de multitarea apropiativa, el comportamiento obtenido es muy similar para la amplia mayoría de los procesos. Incluso para procesos muy largos, este algoritmo no los penaliza mucho más allá de lo que lo haría la ronda, y obtiene mejores promedios de forma consistente porque, al despachar primero los procesos más cortos, mantiene la lista de procesos pendientes corta, lo que lleva naturalmente a menores índices de penalización.

25.4.2. El más penalizado a continuación (HPRN, *highest penalty ratio next*)

En un sistema que no cuenta con multitarea apropiativa, las alternativas presentadas hasta ahora resultan invariablmente injustas: El uso de FCFS favorece los procesos largos, y el uso de SPN los cortos. Un intento de llegar a un algoritmo más balanceado es HPRN.

Todo proceso inicia su paso por la cola de procesos listos con un valor de penalización $P = 1$. Cada vez que es obligado a esperar un tiempo w por otro proceso, P se actualiza como $P = \frac{w+t}{t}$. El proceso que se elige como activo será el que tenga mayor P. Mientras $\rho < 1$, HPRN evitará que incluso los procesos más largos sufran inanición.

En los experimentos realizados por Finkel, HPRN se sitúa siempre en un punto medio entre FCFS y SPN; su principal desventaja se presenta conforme crece la cola de procesos listos, ya que P tiene que calcularse para cada uno de ellos cada vez que el despachador toma una decisión.

25.5. Ronda egoísta (SRR, *selfish round robin*)

Este método busca favorecer los procesos que ya han pasado tiempo ejecutando que a los recién llegados. De hecho, los nuevos procesos no son programados directamente para su ejecución, sino que se les forma en la cola de *procesos nuevos*, y se avanza únicamente con la cola de *procesos aceptados*.

Para SRR se emplean los parámetros a y b, ajustables según las necesidades del sistema. a indica el ritmo según el cual se incrementará la prioridad de los procesos de la cola de *procesos nuevos*, y b el ritmo del incremento de prioridad para los *procesos aceptados*. Cuando la prioridad de un proceso nuevo *alcanza* a la prioridad de un proceso aceptado, el nuevo se vuelve aceptado. Si la cola de procesos aceptados queda vacía, se acepta el proceso nuevo con mayor prioridad. El comportamiento de SRR con los procesos ejemplo es:

Proceso	Tiempo de llegada	t	Inicio	Fin	T	E	P
A	0	3	0	4	4	1	1.3
B	1	5	2	10	9	4	1.8
C	3	2	6	9	6	4	3.0
D	9	5	10	15	6	1	1.2
E	12	5	15	20	8	3	1.6
Promedio		4			6.6	2.6	1.79

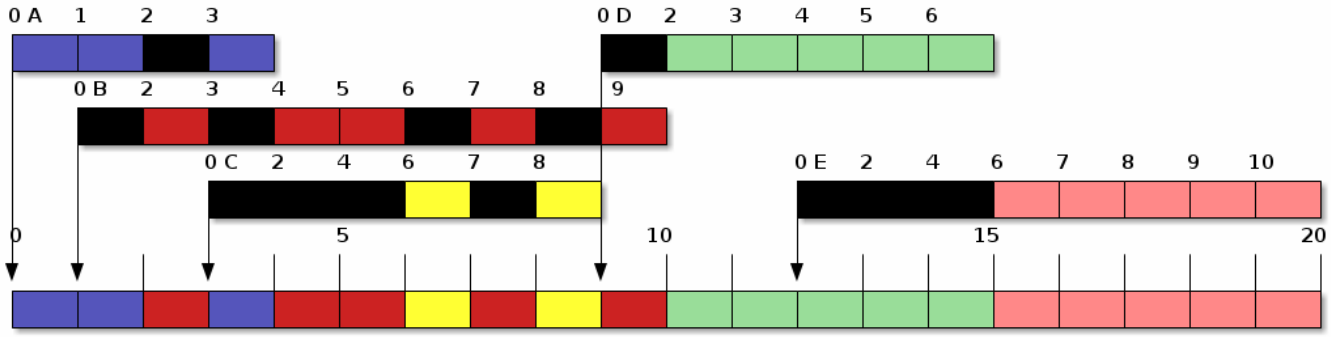


Figura 25.6: Ronda egoísta (SRR) con $a = 2$ y $b = 1$.

Mientras $\frac{b}{a} < 1$, la prioridad de un proceso entrante eventualmente alcanzará a la de los procesos aceptados, y comenzará a ejecutarse. Mientras el control va alternando entre dos o más procesos, la prioridad de todos ellos será la misma (esto es, son despachados efectivamente por una simple ronda).

Incluso cuando $\frac{b}{a} \geq 1$, el proceso en ejecución terminará, y B será aceptado. En este caso, este esquema se convierte en FCFS.

Si $\frac{b}{a} = 0$ (esto es, si $b = 0$), los procesos recién llegados serán aceptados inmediatamente, con lo cual se convierte en una ronda. Mientras $0 < \frac{b}{a} < 1$, la ronda será *relativamente egoísta*, dándole entrada a los nuevos procesos incluso si los que llevan mucho tiempo ejecutando son muy largos (y, por tanto, su prioridad es muy alta).

25.6. Retroalimentación multinivel (FB, multilevel feedback)

El mecanismo descrito en la sección anterior, la *ronda egoísta*, introdujo el concepto de tener no una sino varias colas de procesos, que recibirán diferente tratamiento. Este mecanismo es muy poderoso, y se emplea en prácticamente todos los planificadores en uso hoy en día. Antes de abordar el esquema de retroalimentación multinivel, conviene presentar cómo opera un sistema con múltiples colas de prioridad.

La figura 25.7 ilustra cómo se presentaría una situación bajo esta lógica: el sistema hipotético tiene cinco colas de prioridad, y siete procesos listos para ser puestos en ejecución. Puede haber colas vacías, como en este

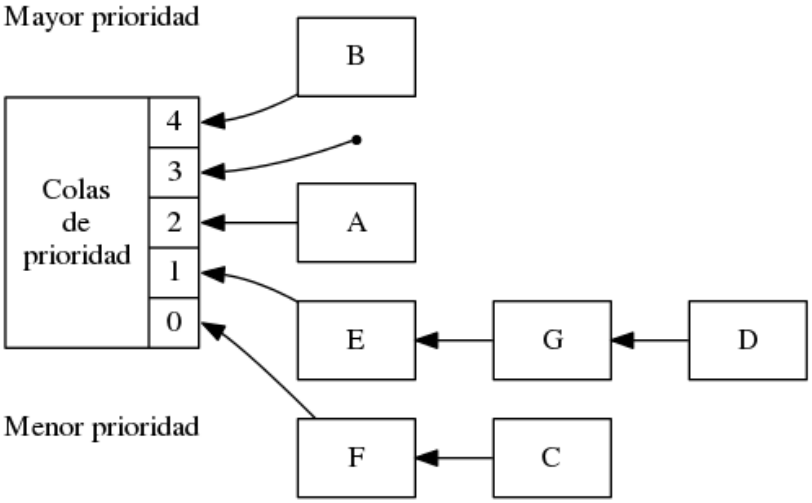


Figura 25.7: Representación de un sistema con cinco colas de prioridad y siete procesos listos.

caso la 3. Dado que la cola de mayor prioridad es la 0, el planificador elegirá únicamente entre los procesos que están *formados* en ella: *F* o *C*. Sólo cuando estos procesos terminen (o sean enviados a alguna otra cola), el planificador continuará con aquellos que estén en las siguientes colas.

La *retroalimentación multinivel* basa su operación en más de una cola —pero en este caso, todas ellas tendrán el mismo tratamiento *general*, distinguiéndose sólo por su nivel de *prioridad*, C_0 a C_n. El despachador elegirá para su ejecución al proceso que esté al frente de la cola de mayor prioridad que tenga algún proceso esperando C_i, y tras un número predeterminado de ejecuciones, lo *degrada* a la cola de prioridad inmediata inferior C_{i+1}.

El mecanismo de retroalimentación multinivel favorece los procesos cortos, dado que terminarán sus tareas sin haber sido marcados como de prioridades inferiores.

La ejecución del juego de datos con que han sido presentados los algoritmos anteriores bajo este esquema da los siguientes resultados:

Proceso	Tiempo de llegada	*t*	Inicio	Fin	*T*	*E*	*P*
A	0	3	0	7	7	4	1.7
B	1	5	1	18	17	12	3.4
C	3	2	3	6	3	1	1.5
D	9	5	9	19	10	5	2.0
E	12	5	12	20	8	3	1.6
Promedio		4			9	5	2.04

Dado que ahora hay que representar la cola en la que está cada uno de los procesos, en la figura 25.8 se presenta sobre cada una de las líneas de proceso la prioridad de la cola en que se encuentra antes del *quantum* a iniciar.

Llama la atención que prácticamente todos los números apuntan a que esta es una peor estrategia que las presentadas anteriormente— los únicos procesos beneficiados en esta ocasión son los recién llegados, que podrán avanzar al principio, mientras los procesos más largos serán castigados y podrán eventualmente (a mayor ρ) enfrentar inanición.

Sin embargo, esta estrategia permite ajustar dos variables: una es la cantidad de veces que un proceso debe

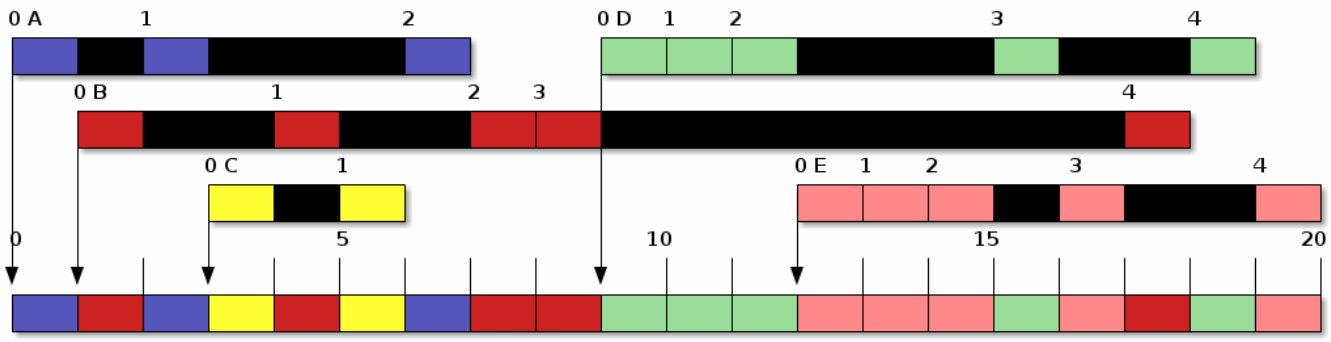


Figura 25.8: Retroalimentación multinivel (FB) básica.

ser ejecutado antes de ser *degradado* a la prioridad inferior, y la otra es la duración del *quantum* asignado a las colas subsecuentes.

Otro fenómeno digno a comentar es el que se presenta a los *ticks* 8, 10, 11, 13 y 14: el despachador interrumpe la ejecución del proceso activo, para volver a cedérsela. Esto ocurre porque, efectivamente, concluyó su *quantum*; idealmente, el despachador se dará cuenta de esta situación de inmediato y no iniciará un cambio de contexto *al mismo proceso*. En caso contrario, el trabajo perdido por gasto administrativo se vuelve innecesariamente alto.

El panorama cambia al ajustar estas variables: si se elige un *quantum* de $2^n q$, donde n es el identificador de cola y q la longitud del *quantum* base, un proceso largo será detenido por un cambio de contexto al llegar a q, $3q$, $7q$, $15q$, etc., lo que llevará al número total de cambios de contexto a $\log(\frac{t(p)}{q})$, lo cual resulta atractivo frente a los $\frac{t(p)}{q}$ cambios de contexto que tendría bajo un esquema de ronda.

Una vez efectuados estos ajustes, la evaluación del mismo juego de procesos bajo retroalimentación multinivel con un *quantum* que sigue un incremento exponencial produce el siguiente resultado:

Proceso	Tiempo de llegada	t	Inicio	Fin	T	E	P
A	0	3	0	4	4	1	1.3
B	1	5	1	10	9	4	1.8
C	3	2	4	8	5	3	2.5
D	9	5	10	18	9	4	1.8
E	12	5	13	20	8	3	1.6
Promedio		4			7	3	1.8

Los promedios de tiempos de terminación, respuesta, espera y penalización para este conjunto de procesos resultan mejores incluso que los de la ronda.

En este caso, a pesar de que esta estrategia favorece a los procesos recién llegados, al *tick* 3, 9 y 10, llegan nuevos procesos, pero a pesar de estar en la cola de mayor prioridad, no son puestos en ejecución, dado que llegaron a la mitad del *quantum* (largo) de otro proceso.

Típicamente se emplean incrementos mucho más suaves, y de crecimiento más controlado, como nq o inlcuso $q\log(n)$, dado que en caso contrario un proceso muy largo podría causar muy largas inaniciones para el resto del sistema.

Para evitar la inanición, puede considerarse también la retroalimentación en sentido inverso: si un proceso largo fue *degradado* a la cola C_P y pasa determinado tiempo sin recibir servicio, puede *promoverse* de nuevo a la cola C_{P-1} para que no sufra inanición.

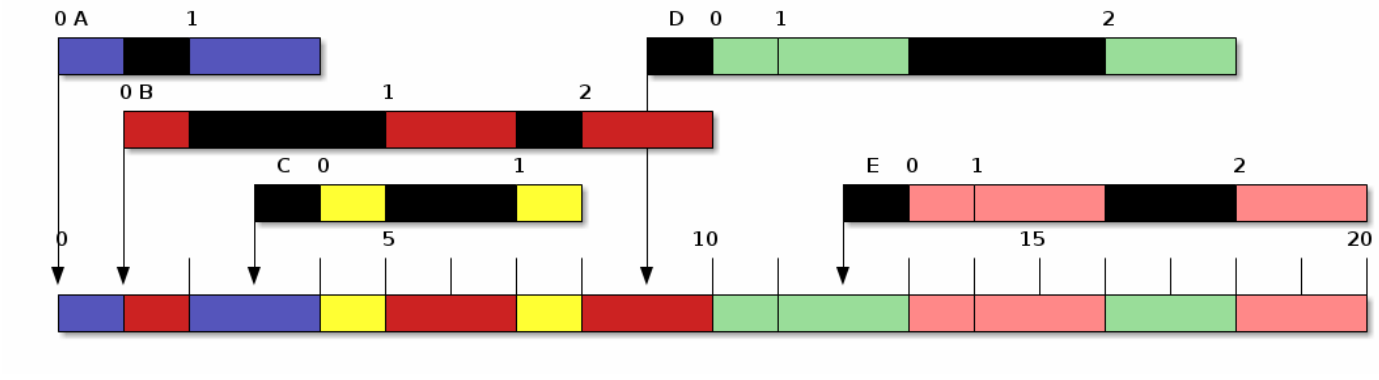


Figura 25.9: Retroalimentación multinivel (FB) con q exponencial.

Hoy en día, muchos de los principales sistemas operativos operan bajo diferentes versiones de retroalimentación multinivel, y típicamente con hasta decenas de colas.

25.7. Lotería

Los mecanismos hasta aquí descritos vienen con largas décadas de desarrollo. Uno de los últimos algoritmos que ha sido ampliamente difundido en unirse a esta lista es el de *planificación por lotería* (Waldspurger y Weihl 1994).

Bajo el esquema de la *lotería*, cada proceso tiene un número determinado de boletos, y cada boleto le representa una oportunidad de jugar a la lotería. Cada vez que el planificador tiene que elegir el siguiente proceso a poner en ejecución, elige un número al azar[4], y otorga el siguiente quantum al proceso que tenga el boleto ganador. El boleto ganador *no es retirado*, esto es, la probabilidad de que determinado proceso sea puesto en ejecución no varía entre invocaciones sucesivas del planificador.

Las prioridades pueden representarse en este esquema de forma muy sencilla: un proceso al que se le quiere dar mayor prioridad simplemente tendrá más boletos; si el proceso A tiene 20 boletos y el proceso B tiene 60, será tres veces más probable que el siguiente turno toque a B que a A.

El esquema de planificación por lotería considera que los procesos puedan cooperar entre sí: si B estuviera esperando un resultado de A, podría transferirle sus boletos para aumentar la probabilidad de que sea puesto en ejecución.

A pesar de su simplicidad, el esquema de planificación por lotería resulta justo, tanto a procesos cortos, como a largos, y presenta una degradación muy suave incluso en entornos de saturación. Claro, al derivar de un proceso aleatorio, resulta imposible presentar una comparación de este mecanismo abordado previamente.

25.8. Esquemas híbridos

En líneas generales, los siete algoritmos presentados pueden clasificarse sobre dos discriminadores primarios: si están pensados para emplearse en multitarea cooperativa o apropiativa, y si emplean información *intrínseca* a los procesos evaluados o no lo hacen, esto es, si un proceso es tratado de distinta forma dependiendo de su historial de ejecución.

[4]Si bien operar un generador de números aleatorios en estricto sentido sería demasiado caro para un proceso que se ejecuta decenas o cientos de veces por segundo, para *jugar* a la lotería es suficiente emplear un generador débil pseudoaleatorio. El artículo en que este mecanismo fue presentado presenta la implementación del algoritmo Park-Miller, $S' = (A \times S) mod (2^{31} - 1)$ con $A = 16\,807$, implementado en 12 instrucciones de procesador RISC.

Cuadro 25.1: Caracterización de los mecanismos de planificación a corto plazo.

	NO CONSIDERA INTRÍNSECA	CONSIDERA INTRÍNSECA
COOPERATIVA	Primero llegado primero servido (FCFS)	Proceso más corto (SPN), Proceso más penalizado (HPRN)
PREVENTIVA	Ronda (RR) Lotería	Proceso más corto apropiativo (PSPN), Retroalimentación (FB), Ronda egoísta (SRR)

Ahora bien, estas características primarias pueden ser empleadas en conjunto, usando diferentes algoritmos a diferentes niveles, o cambiándolos según el patrón de uso del sistema, aprovechando de mejor manera sus bondades y logrando evitar sus deficiencias. A continuación, algunos ejemplos de esquemas híbridos.

25.8.1. Algoritmo por cola dentro de FB

Al introducir varias colas, se abre la posibilidad de que cada una de ellas siga un esquema diferente para elegir cuál de sus procesos está a la cabeza. En los ejemplos antes presentados, todas las colas operaban siguiendo una ronda, pero podría considerarse, por ejemplo, que parte de las colas sean procesadas siguiendo una variación de PSPN que *empuje* a los procesos más largos a colas que les puedan dar atención con menor número de interrupciones (incluso sin haberlos ejecutado aún).

Podría emplearse un esquema SRR para las colas de menor prioridad, siendo que ya tienen procesos que han esperado mucho tiempo para su ejecución, para que –sin que repercutan en el tiempo de respuesta de los procesos cortos que van entrando a las colas superiores– terminen lo antes posible su ejecución.

25.8.2. Métodos dependientes del estado del sistema

Los parámetros de operación pueden variar también dependiendo del estado actual del sistema, e incluso tomando en consideración valores externos al despachador. Algunas ideas al respecto son:

- Si los procesos listos son *en promedio* no muy largos, y el valor de saturación es bajo ($\rho < 1$), optar por los métodos que menos sobrecarga administrativa signifiquen, como FCFS o SPN (o, para evitar los peligros de la multitarea cooperativa, un RR con un *quantum* muy largo). Si el despachador observa que la longitud de la cola excede un valor determinado (o *muestra una tendencia* en ese sentido, al incrementarse ρ), tendrá que cambiar a un mecanismo que garantice una mejor distribución de la atención, como un RR con *quantum* corto o PSPN.

- Usar un esquema simple de ronda. La duración de un *quantum* puede ser ajustada periódicamente (a cada cambio de contexto, o como un cálculo periódico), para que la duración del siguiente *quantum* dependa de la cantidad de procesos en espera en la lista, $Q = \frac{q}{n}$.

  Si hay pocos procesos esperando, cada uno de ellos recibirá un *quantum* más largo, reduciendo la cantidad de cambios de contexto. Si hay muchos, cada uno de ellos tendrá que esperar menos tiempo para comenzar a liberar sus pendientes.

Claro está, la duración de un *quantum* no debe reducirse más allá de cierto valor mínimo, definido según la realidad del sistema en cuestión, dado que podría aumentar demasiado la carga burocrática.

- Despachar los procesos siguiendo una ronda, pero asignarles una duración de *quantum* proporcional a su prioridad externa (fijada por el usuario). Un proceso de mayor prioridad ejecutará *quantums* más largos.

- *Peor servicio a continuación* (WSN, *worst service next*). Es una generalización sobre varios de los mecanismos mencionados; su principal diferencia respecto a HPRN es que no sólo se considera *penalización* el tiempo que ha pasado esperando en la cola, sino que se considera el número de veces que ha sido interrumpido por el temporizador o su prioridad externa, y se considera (puede ser a favor o en contra) el tiempo que ha tenido que esperar por E/S u otros recursos. El proceso que ha sufrido del *peor servicio* es seleccionado para su ejecución, y si varios empatan, se elige uno en ronda.

  La principal desventaja de WSN es que, al considerar tantos factores, el tiempo requerido, por un lado, para recopilar todos estos datos y, por otro, calcular el peso que darán a cada uno de los procesos implicados, puede repercutir en el tiempo global de ejecución. Es posible acudir a WSN periódicamente (y no cada vez que el despachador es invocado) para que reordene las colas según criterios generales, y abanzar sobre dichas colas con algoritmos más simples, aunque esto reduce la velocidad de reacción ante cambios de comportamiento.

- Algunas versiones históricas de Unix manejaban un esquema en que la prioridad especificada por el usuario[5] era matizada y re-evaluada en el transcurso de su ejecución.

  Periódicamente, para cada proceso se calcula una prioridad *interna*, que depende de la prioridad *externa* (especificada por el usuario) y el tiempo consumido recientemente por el proceso. Conforme éste recibe mayor tiempo de procesador, esta última cantidad decrece, y aumenta conforme el proceso espera (sea por decisión del despachador o por estar en alguna espera).

  Esta prioridad interna depende también del tamaño de la lista de procesos listos para su ejecución: entre más procesos haya pendientes, más fuerte será la modificación que efectúe.

  El despachador ejecutará al proceso que tenga una mayor prioridad después de realizar estos pasos, decidiendo por ronda en caso de haber empates. Claro está, este algoritmo resulta sensiblemente más caro computacionalmente, aunque más justo, que aquellos sobre los cuales construye.

25.9. Resumiendo

En esta sección se presentan algunos mecanismos básicos de planificación a corto plazo. Como se indica en la parte final, es muy poco común encontrar estos mecanismos en un *estado puro* —normalmente se encuentra una combinación de ellos, con diferentes parámetros según el nivel en el cual se está ejecutando.

25.9.1. Rendimiento ante diferentes cargas de procesos

Los algoritmos de esta sección fueron presentados y comparados ante una determinada carga de procesos. No se puede asumir, sin embargo, que su comportamiento será igual ante diferentes distribuciones: un patrón de trabajo donde predominen los procesos cortos y haya unos pocos procesos largos probablemente se verá mucho más penalizado por un esquema SRR (y mucho más favorecido por un SPN o PSPN) que uno en el cual predominen los procesos largos.

[5] La *lindura*, o *niceness* de un proceso, llamada así por establecerse por medio del comando `nice` al iniciar su ejecución, o `renice` una vez en ejecución

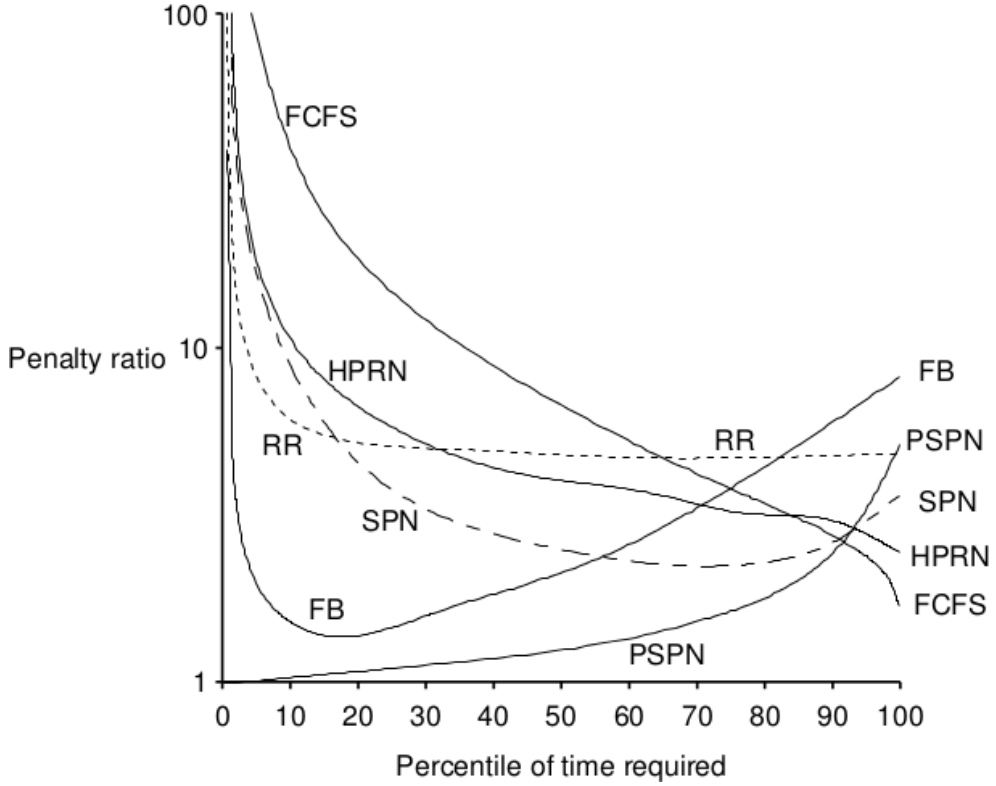


Figura 25.10: Proporción de penalización registrada por cada proceso contra el porcentaje del tiempo que éste requiere (Finkel 1988: 33).

Raphael Finkel realizó estudios bajo diversas cargas de trabajo, buscando comparar de forma *significativa* estos distintos mecanismos. Finkel simuló el comportamiento que estos algoritmos tendrían ante 50 000 procesos generados de forma aleatoria, siguiendo una distribución exponencial, tanto en sus tiempos de llegada, como en su duración en ejecución, y manteniendo como parámetro de equilibrio un nivel de saturación $\rho = 0{,}8 (\alpha = 0{,}8, \beta = 1{,}0)$, obteniendo como resultado las figuras aquí reproducidas (25.10 y 25.11) comparando algunos aspectos importantes de los diferentes despachadores.

25.9.2. Duración mínima del *quantum*

La penalización por cambios de contexto en esquemas apropiativos como la *ronda* puede evitarse empleando *quantums* mayores. Pero abordando la contraparte, ¿qué tan corto tiene sentido que sea un *quantum*? Con el hardware y las estructuras requeridas por los sistemas operativos de uso general disponibles hoy en día, un cambio de contexto requiere del orden de 10 microsegundos (Silberschatz, Galvin y Gagne 2010: 187), por lo que incluso con el *quantum* de 10 ms (el más corto que manejan tanto Linux como Windows), representa apenas la milésima parte del tiempo efectivo de proceso.

Una estrategia empleada por Control Data Corporation para la CDC6600 (comercializada a partir de 1964, y diseñada por Seymour Cray) fue emplear hardware especializado que permitiera efectivamente *compartir el procesador*: un sólo procesador tenía 10 *juegos* de registros, permitiéndole alternar entre 10 procesos con un *quantum* efectivo igual a la velocidad del reloj. A cada paso del reloj, el procesador cambiaba el juego de registros. De este modo, un sólo procesador de muy alta velocidad para su momento (1 MHz) aparecía ante las aplicaciones como 10 procesadores efectivos, cada uno de 100 KHz, reduciendo los costos al implementar sólamente una vez cada una de las unidades funcionales. Puede verse una evolución de esta idea retomada

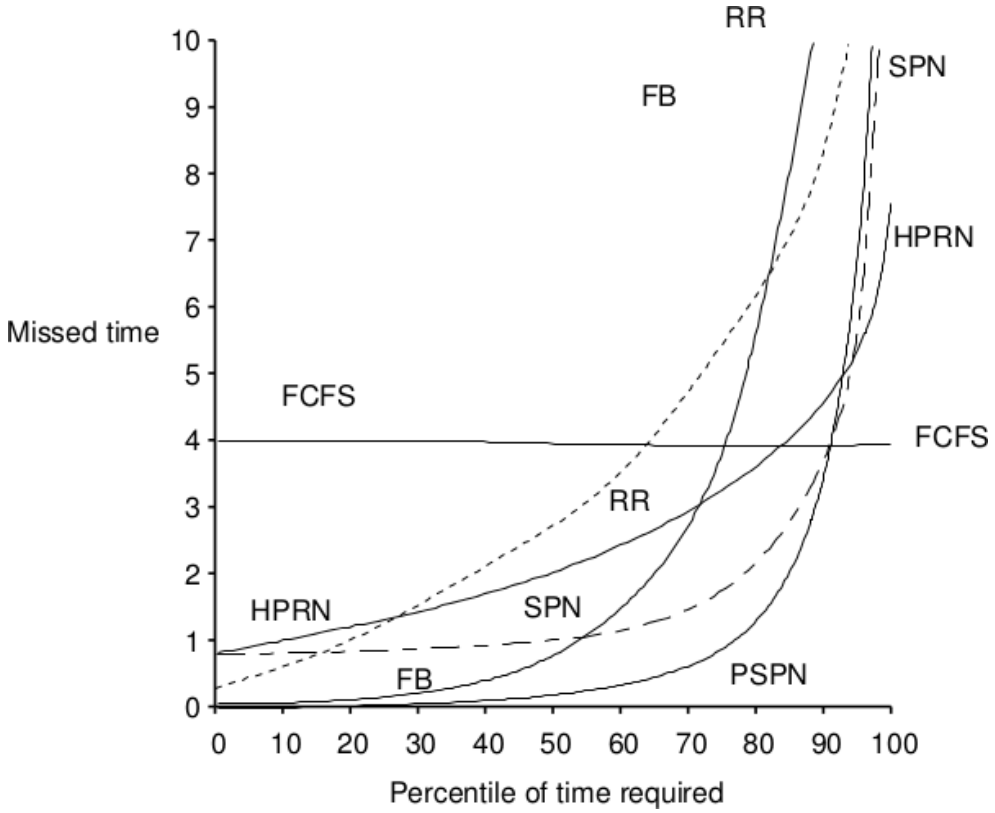


Figura 25.11: Tiempo *perdido* contra porcentaje de tiempo requerido por proceso (Finkel 1988: 34).

hacia mediados de la década del 2000 en los procesadores que manejan hilos de ejecución.[6]

Esta arquitectura permitía tener multitarea real sin tener que realizar cambios de contexto, sin embargo, al tener un *nivel de concurrencia* fijo establecido en hardware no es tan fácil adecuar a un entorno cambiante, con picos de ocupación.

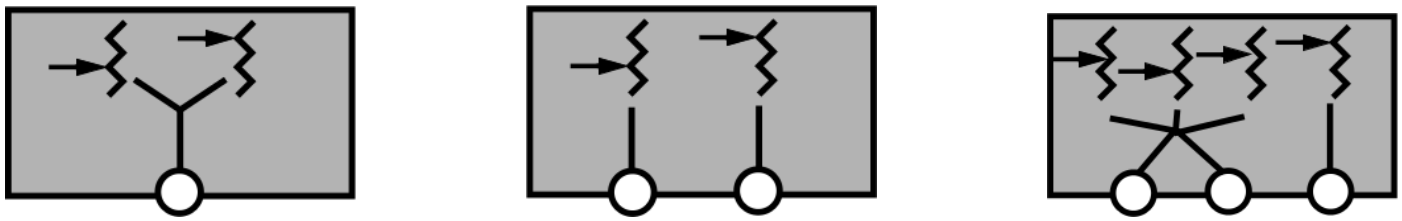
Capítulo 26

Planificación de hilos

Ahora bien, tras centrar toda la presente discusión en los procesos, ¿cómo caben los *hilos* en este panorama? Depende de cómo éstos son *mapeados* a procesos a ojos del planificador.

Como fue expuesto en la sección 20.1, hay dos clases principales de hilo: los *hilos de usuario* o *hilos verdes*,

[6] Aunque la arquitecura de la CDC6600 era *plenamente superescalar*, a diferencia de los procesadores *Hyperthreading*, que será abordada brevemente en la sección 27.4, en que para que dos instrucciones se ejecuten simultáneamente deben ser de naturalezas distintas, no requiriendo ambas de la misma *unidad funcional* del CPU. El procesador de la CDC6600 no manejaba *pipelines*, sino que cada ejecución empleaba al CPU completo



(a) Muchos a uno (b) Uno a uno (c) Muchos a muchos

Figura 26.1: Tres modelos de mapeo de hilos a procesos. (Imágenes: Beth Plale; véase sección 29.3).

que son completamente gestionados dentro del proceso y sin ayuda del sistema operativo, y los *hilos de núcleo* o *hilos de kernel*, que sí son gestionados por el sistema operativo como si fueran procesos. Partiendo de esto, hay tres modelos principales de mapeo:

Muchos a uno Muchos hilos son agrupados en un sólo proceso. Los *hilos verdes* entran en este supuesto: para el sistema operativo, hay un sólo proceso; mientras tiene la ejecución, éste se encarga de repartir el tiempo entre sus hilos.

Bajo este modelo, si bien el código escrito es más portable entre diferentes sistemas operativos, los hilos no aprovechan *realmente* al paralelismo, y todos los hilos pueden tener que bloquearse cuando uno solo de ellos realiza una llamada *bloqueante* al sistema.

La figura 26.1(a) ilustra este modelo.

Uno a uno Cada hilo es ejecutado como un *proceso ligero* (*lightweight process* o LWP); podría dar la impresión de que este esquema desperdicia la principal característica de los hilos, que es una mayor sencillez y rapidez de inicialización que los procesos, sin embargo, la información de estado requerida para crear un LWP es mucho menor que la de un proceso regular, y mantiene como ventaja que los hilos continúan compartiendo su memoria, descriptores de archivos y demás estructuras.

Este mecanismo permite a los hilos aprovechar las ventajas del paralelismo, pudiendo ejecutarse cada hilo en un procesador distinto, y como única condición, el sistema operativo debe poder implementar los LWP.

La esquematización de este modelo puede apreciarse en la figura 26.1(b).

Muchos a muchos Este mecanismo permite que hayan hilos de ambos modelos: permite *hilos unidos* (*bound threads*), en que cada hilo corresponde a un (y solo un) LWP, y de *hilos no unidos* (*unbound threads*), de los cuales *uno o más* estarán mapeados a cada LWP.

El esquema *muchos a muchos* proporciona las principales características de ambos esquemas; en caso de ejecutarse en un sistema que no soporte más que el modelo *uno a muchos*, el sistema puede caer en éste como *modo degradado*. Este modelo se presenta en la figura 26.1(c).

No se detalla en el presente texto respecto a los primeros —cada marco de desarrollo o máquina virtual (véase la sección F.1) que emplee *hilos de usuario* actuará cual sistema operativo ante ellos, probablemente con alguno de los mecanismos ilustrados anteriormente.

26.1. Los hilos POSIX (`pthreads`)

La clasificiación recién presentada de modelos de mapeo entre hilos y procesos se refleja aproximadamente en la categorización denominada el *ámbito de contención* de los hilos POSIX (`pthreads`).

Hay dos enfoques respecto a la *contención* que deben tener los hilos, esto es: en el momento que un proceso separa su ejecución en dos hilos, ¿debe cada uno de estos recibir la misma atención que recibiría un proceso completo?

Ámbito de contención de proceso (*Process Contention Scope*, PCS; en POSIX, `PTHREAD_SCOPE_PROCESS`). Una respuesta es que todos los hilos deben ser atendidos sin exceder el tiempo que sería asignado a un solo proceso. Un proceso que consta de varios hilos siguiendo el modelo *muchos a uno*, o uno que *multiplexa* varios *hilos no unidos* bajo un modelo *muchos a muchos*, se ejecuta bajo este ámbito.

Ámbito de contención de sistema (*System Contention Scope*, SCS; en POSIX, `PTHREAD_SCOPE_SYSTEM`). Este ámbito es cuando, en contraposición, cada hilo es visto por el planificador como un proceso independiente; este es el ámbito en el que se ejecutarían los hilos bajo el modelo *uno a uno*, o cada uno de los *hilos unidos* bajo un modelo *muchos a muchos*, dado que los hilos son tratados, para propósitos de planificación, cual procesos normales.

La definición de `pthreads` apunta a que, si bien el programador puede solicitar que sus hilos sean tratados bajo cualquiera de estos procesos, una implementación específica puede presentar ambos o solo uno de los ámbitos. Un proceso que solicite que sus hilos sean programados bajo un ámbito no implementado serán ejecutados bajo el otro, notificando del error (pero permitiendo continuar con la operación).

Las implementaciones de `pthreads` tanto en Windows como en Linux sólo consideran SCS.

Respecto a los otros aspectos mencionados en este capítulo, la especificación `pthreads` incluye funciones por medio de las cuales el programador puede solicitar al núcleo la prioridad de cada uno de los hilos por separado (`pthread_setschedprio`) e incluso pedir el empleo de determinado algoritmo de planificación (`sched_setscheduler`).

Capítulo 27

Planificación de multiprocesadores

Hasta este punto, el enfoque de este capítulo se ha concentrado en la planificación asumiendo un solo procesador. Del mismo modo que lo que se ha visto hasta este momento, no hay una sola estrategia que pueda ser vista como superior a las demás en todos los casos.

Para trabajar en multiprocesadores, puede mantenerse una sola lista de procesos e ir despachándolos a cada uno de los procesadores como unidades de ejecución equivalentes e idénticas, o pueden mantenerse listas separadas de procesos. A continuación se presentan algunos argumentos respecto a estos enfoques.

27.1. Afinidad a procesador

En un entorno multiprocesador, después de que un proceso se ejecutó por cierto tiempo, tendrá parte importante de sus datos copiados en el caché del procesador en el que fue ejecutado. Si el despachador decidiera

lanzarlo en un procesador que no compartiera dicho caché, estos datos tendrían que ser *invalidados* para mantener la coherencia, y muy probablemente (por *localidad de referencia*) serían vueltos a cargar al caché del nuevo procesador.

Los procesadores actuales normalmente tienen disponibles varios niveles de caché; si un proceso es migrado entre dos núcleos del mismo procesador, probablemente solo haga falta invalidar los datos en el caché más interno (L1), dado que el caché en chip (L2) es compartido entre los varios núcleos del mismo chip; si un proceso es migrado a un CPU físicamente separado, será necesario invalidar también el caché en chip (L2), y mantener únicamente el del controlador de memoria (L3).

Pero dado que la situación antes descrita varía de una computadora a otra, no se puede enunciar una regla general —más allá de que el sistema operativo debe conocer cómo están estructurados los diversos procesadores que tiene a su disposición, y buscar realizar las migraciones *más baratas*, aquellas que tengan lugar entre los procesadores más cercanos.

Resulta obvio por esto que un proceso que fue ejecutado en determinado procesador vuelva a hacerlo en el mismo, esto es, el proceso *tiene afinidad* por cierto procesador. Un proceso que *preferentemente* será ejecutado en determinado procesador se dice que *tiene afinidad suave* por él, pero determinados patrones de carga (por ejemplo, una mucho mayor cantidad de procesos afines a cierto procesador que a otro, saturando su cola de procesos listos, mientras que el segundo procesador tiene tiempo disponible) pueden llevar a que el despachador decida activarlo en otro procesador.

Por otro lado, algunos sistemas operativos ofrecen la posibilidad de declarar *afinidad dura*, con lo cual se *garantiza* a un proceso que siempre será ejecutado en un procesador, o en un conjunto de procesadores.

Un entorno NUMA, por ejemplo, funcionará mucho mejor si el sistema que lo emplea maneja tanto un esquema de afinidad dura como algoritmos de asignación de memoria que le aseguren que un proceso siempre se ejecutará en el procesador que tenga mejor acceso a sus datos.

27.2. Balanceo de cargas

En un sistema multiprocesador, la situación ideal es que todos los procesadores estén despachando trabajos a 100% de su capacidad. Sin embargo, ante una definición tan rígida, la realidad es que siempre habrá uno o más procesadores con menos de 100% de carga, o uno o más procesadores con procesos encolados y a la espera, o incluso ambas situaciones.

La divergencia entre la carga de cada uno de los procesadores debe ser lo más pequeña posible. Para lograr esto, se pueden emplear esquemas de *balanceo de cargas*: algoritmos que analicen el estado de las colas de procesos y, de ser el caso, transfieran procesos entre las colas para homogeneizarlas. Claro está, el balanceo de cargas puede actuar precisamente en sentido contrario de la afinidad al procesador y, efectivamente, puede reubicar a los procesos con afinidad suave.

Hay dos estrategias primarias de balanceo: por un lado, la migración activa o migración *por empuje* (*push migration*) consiste en una tarea que ejecuta como parte del núcleo y periódicamente revisa el estado de los procesadores, y en caso de encontrar un desbalance mayor a cierto umbral, *empuja* a uno o más procesos de la cola del procesador más ocupado a la del procesador más libre. Linux ejecuta este algoritmo cada 200 milisegundos.

Por otro lado, está la migración pasiva o migración *por jalón* (*pull migration*). Cuando algún procesador queda sin tareas pendientes, ejecuta al proceso especial *desocupado* (*idle*). Ahora, el proceso *desocupado* no significa que el procesador detenga su actividad —ese tiempo puede utilizarse para ejecutar tareas del núcleo. Una de esas tareas puede consistir en averiguar si hay procesos en espera en algún otro de los procesadores, y de ser así, *jalarlo* a la cola de este procesador.

Ambos mecanismos pueden emplearse –y normalmente lo hacen– en el mismo sistema. Los principales sistemas operativos modernos emplean casi siempre ambos mecanismos.

Como sea, debe mantenerse en mente que todo balanceo de cargas que se haga entre los procesadores conllevará una penalización en términos de afinidad al CPU.

27.3. Colas de procesos: ¿una o varias?

En los puntos relativos al multiprocesamiento hasta ahora abordados se parte del supuesto que hay una cola de procesos listos por cada procesador. Si, en cambio, hubiera una cola global de procesos listos de la cual el siguiente proceso a ejecutarse fuera asignándose al siguiente procesador, fuera éste cualquiera de los disponibles, podría ahorrarse incluso elegir entre una estrategia de migración *por empuje* o *por jalón* —mientras hubiera procesos pendientes, éstos serían asignados al siguiente procesador que tuviera tiempo disponible.

El enfoque de una sola cola, sin embargo, no se usa en ningún sistema en uso amplio. Esto es en buena medida porque un mecanismo así haría mucho más difícil mantener la afinidad al procesador y restaría flexibilidad al sistema completo.

27.4. Procesadores con soporte a *hilos hardware*

El término de *hilos* como abstracción general de algo que se ejecuta con mayor frecuencia y dentro de un mismo proceso puede llevar a una confusión, dado que en esta sección se tocan dos temas relacionados. Para esta subsección en particular, se hace referencia a los *hilos en hardware* (en inglés, *hyperthreading*) que forman parte de ciertos procesadores, ofreciendo al sistema una *casi* concurrencia adicional.

Conforme han subido las frecuencias de reloj en el cómputo más allá de lo que permite llevar al sistema entero como una sola unidad, una respuesta recurrente ha sido incrementar el paralelismo. Y esto no solo se hace proveyendo componentes completos adicionales, sino separando las *unidades funcionales* de un procesador.

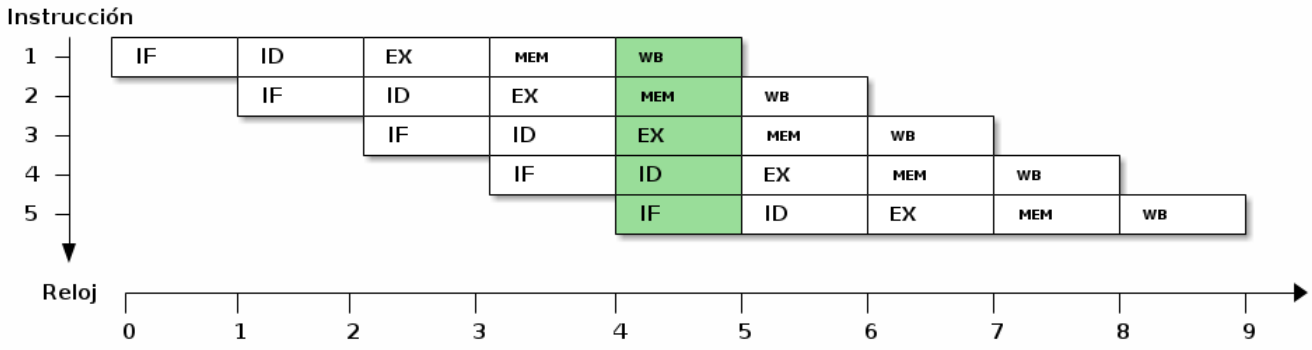


Figura 27.1: Descomposición de una instrucción en sus cinco pasos clásicos para organizarse en un *pipeline*.

El flujo de una sola instrucción a través de un procesador simple como el MIPS puede ser dividido en cinco secciones principales, creando una estructura conocida como *pipeline* (tubería). Idealmente, en todo momento habrá una instrucción diferente ejecutando en cada una de las secciones del procesador, como lo ilustra la figura 27.1. El *pipeline* de los procesadores MIPS clásicos se compone de las siguientes secciones:

- Recuperación de la instrucción (*Instruction Fetch*, `IF`).
- Decodificación de la instrucción (*Instruction Decode*, `ID`).
- Ejecución (*Execution*, `EX`).

- Acceso a datos (MEM).
- Almacenamiento (Writeback, WB).

La complejidad de los procesadores actuales ha crecido ya por encima de lo aquí delineado (el Pentium 4 tiene más de 20 etapas), sin embargo se presenta esta separación como base para la explicación. Un procesador puede iniciar el procesamiento de una instrucción cuando la siguiente apenas avanzó la quinta parte de su recorrido —de este modo, puede lograr un paralelismo interno, manteniendo idealmente siempre ocupadas a sus partes funcionales.

Sin embargo, se ha observado que un hay patrones recurrentes que intercalan operaciones que requieren servicio de diferentes componentes del procesador, o que requieren de la inserción de *burbujas* porque una unidad es más lenta que las otras —lo cual lleva a que incluso empleando *pipelines*, un procesador puede pasar hasta 50% del tiempo esperando a que haya datos disponibles solicitados a la memoria.

Para remediar esto, varias de las principales familias de procesadores presentan a un mismo *núcleo* de procesador como si estuviera compuesto de dos o más *hilos hardware* (conocidos en el mercado como *hyperthreads*). Esto puede llevar a una mayor utilización del procesador, siguiendo patrones como el ilustrado en la figura 27.2.

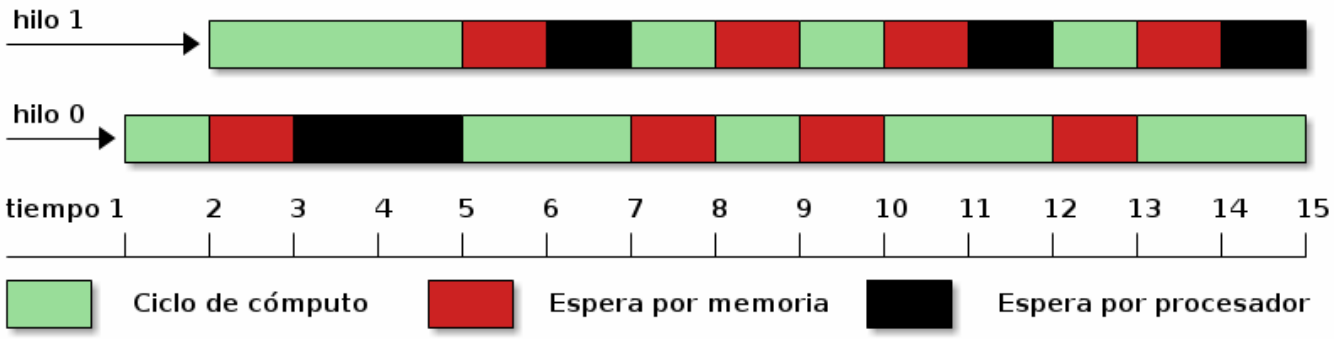


Figura 27.2: Alternando ciclos de cómputo y espera por memoria, un procesador que implementa hilos hardware (*hyperthreading*) es visto por el sistema como dos procesadores lógicos.

Hay que recordar que, a pesar de que se *presenten* como hilos independientes, el rendimiento de cada uno depende de la secuencia particular de instrucciones del otro —no puede esperarse que el incremento en el procesamiento sea de 2x; la figura presenta varios puntos en que un hilo está en *espera por procesador*, dado que el otro está empleando las unidades funcionales que éste requiere.

La planificación de los hilos hardware sale del ámbito del presente material, y este tema se presenta únicamente para aclarar un concepto que probablemente confunda al alumno por su similitud; los hilos en hardware implican cuestiones de complejidad tal como el ordenamiento específico de las instrucciones, predicción de ramas de ejecución, e incluso asuntos relativos a la seguridad, dado que se han presentado *goteos* que permiten a un proceso ejecutando en un hilo *espiar* el estado del procesador correspondiente a otro de los hilos. Para abundar al respecto, el ámbito adecuado podría ser un texto orientado a la construcción de compiladores (ordenamiento de instrucciones, aprovechamiento del paralelismo), o uno de arquitectura de sistemas (estudio del *pipeline*, aspectos del hardware).

Esta estrategia guarda gran similitud, y no puede evitar hacerse el paralelo, con la *compartición de procesador* empleada por la CDC6600, presentada en la sección 25.9.

Capítulo 28

Tiempo real

Todos los esquemas de manejo de tiempo hasta este momento se han enfocado a repartir el tiempo disponible entre todos los procesos que requieren atención. Es necesario también abordar los procesos que *requieren garantías de tiempo*: procesos que para poder ejecutarse deben garantizar el haber tenido determinado tiempo de proceso antes de un tiempo límite. Los procesos con estas características se conocen como *de tiempo real*.

Hay ejemplos de procesos que requieren este tipo de planificación a todo nivel; los más comunes son los controladores de dispositivos y los recodificadores o reproductores de medios (audio, video). La lógica general es la misma. Para agendarse como un proceso con requisitos de tiempo real, éste debe declarar sus requisitos de tiempo (formalmente, *efectuar su reserva de recursos*) al iniciar su ejecución o en el transcurso de la misma. Claro está, siendo que los procesos de tiempo real obtienen una *prioridad* mucho mayor a otros, normalmente se requerirá al iniciar el proceso que éste *declare* que durante parte de su ejecución trabajará con restricciones de tiempo real.

28.1. Tiempo real duro y suave

Supóngase que un dispositivo genera periódicamente determinada cantidad de información y la va colocando en un área determinada de memoria compartida (en un *buffer*). Al inicializarse, su controlador declarará al sistema operativo cuánto tiempo de ejecución le tomará recoger y procesar dicha información, liberando el *buffer* para el siguiente ciclo de escritura del dispositivo, y la frecuencia con que dicha operación tiene que ocurrir.

En un sistema capaz de operar con garantías de tiempo real, si el sistema operativo puede *garantizar* que en ese intervalo le otorgará al proceso en cuestión suficiente tiempo para procesar esta información, el proceso se ejecuta; en caso contrario, recibe un error *antes de que esto ocurra* por medio del cual podrá alertar al usuario.

Los sistemas en que el tiempo máximo es garantizable son conocidos como de *tiempo real duro*.

La necesidad de atención en tiempo real puede manejarse *periódica* (por ejemplo, *requiero del procesador por 30 ms cada segundo*), o *aperiódica*, por ocurrencia única (*necesito que este proceso, que tomará 600 ms, termine de ejecutarse en menos de 2 segundos*).

Realizar una reserva de recursos requiere que el planificador sepa con certeza cuánto tiempo toma realizar las tareas de sistema que ocurrirán en el periodo en cuestión. Cuando entran en juego algunos componentes de los sistemas de propósito general que tienen una latencia con variaciones impredecibles (como el almacenamiento en disco o la memoria virtual) se vuelve imposible mantener las garantías de tiempo ofrecidas. Por esta razón, en un sistema operativo de propósito general empleando hardware estándar *no es posible* implementar tiempo real duro.

Para solventar necesidades como las expresadas en sistemas de uso general, el *tiempo real suave* sigue requiriendo que los procesos críticos reciban un trato prioritario por encima de los processos comunes; agendar un proceso con esta prioridad puede llevar a la inanición de procesos de menor prioridad y un comportamiento que bajo ciertas métricas resultaría *injusto*. Un esquema de tiempo real suave puede implementarse mediante un esquema similar al de la *retroalimentación multinivel*, con las siguientes particularidades:

- La cola de tiempo real recibe prioridad sobre todas las demás colas.

- La prioridad de un proceso de tiempo real *no se degrada* conforme se ejecuta repetidamente.
- La prioridad de los demás procesos *nunca llegan a subir* al nivel de tiempo real por un proceso automático (aunque sí puede hacerse por una llamada explícita).
- La latencia de despacho debe ser mínima.

Casi todos los sistemas operativos en uso amplio hoy en día ofrecen facilidades básicas de tiempo real suave.

28.2. Sistema operativo interrumpible (*prevenible*)

Para que la implementación de tiempo real suave sea apta para estos requisitos es necesario modificar el comportamiento del sistema operativo. Cuando un proceso de usuario hace una llamada al sistema, o cuando una interrupción corta el flujo de ejecución, hace falta que el sistema procese completa la rutina que da servicio a dicha solicitud antes de que continúe operando. Se dice entonces que el sistema operativo *no es prevenible* o *no es interrumpible*.

Para lograr que el núcleo pueda ser interrumpido para dar el control de vuelta a procesos de usuario, un enfoque fue el poner *puntos de interrupción* en los puntos de las funciones del sistema donde fuera seguro, tras asegurarse que las estructuras estaban en un estado estable. Esto, sin embargo, no modifica mucho la situación porque estos puntos son relativamente pocos, y es muy difícil reestructurar su lógica para permitir puntos de prevención adicionales.

Otro enfoque es hacer al núcleo entero completamente interrumpible, asegurándose de que, a lo largo de todo su código, todas las modificaciones a las estructuras internas estén protegidas por mecanismos de sincronización, como los estudiados en la sección 21. Este método ralentiza varios procesos del núcleo, pero es mucho más flexible, y ha sido adoptado por los diversos sistemas operativos. Tiene la ventaja adicional de que permite que haya *hilos* del núcleo ejecutando de forma concurrente en todos los procesadores del sistema.

28.3. Inversión de prioridades

Un efecto colateral de que las estructuras del núcleo estén protegidas por mecanismos de sincronización es que puede presentarse la *inversión de prioridades*. Esto es:

- Un proceso A de baja prioridad hace una llamada al sistema, y es interrumpido a la mitad de dicha llamada.
- Un proceso B de prioridad *tiempo real* hace una segunda llamada al sistema, que requiere de la misma estructura que la que tiene bloqueada el proceso A.

Al presentarse esta situación, B se quedará esperando hasta que A pueda ser nuevamente agendado —esto es, un proceso de alta prioridad no podrá avanzar hasta que uno de baja prioridad libere su recurso.

La respuesta introducida por Solaris 2 a esta situación a este fenómeno es la *herencia de prioridades*: todos los procesos que estén accesando (y, por tanto, bloqueando) recursos requeridos por un proceso de mayor prioridad, serán tratados como si fueran de la prioridad de dicho recurso *hasta que terminen de utilizar el recurso en cuestión*, tras lo cual volverán a su prioridad nativa.

Capítulo 29

Ejercicios

29.1. Preguntas de autoevaluación

1. En un sistema interactivo, los procesos típicamente están en ejecución un largo periodo (entre minutos y días), sin embargo, en el transcurso del capítulo estos fueron casi siempre tratados como *procesos cortos*. ¿Qué significa esto? ¿Cuál sería un ejemplo de *proceso largo*?

2. Asumiendo los siguientes procesos:

Proceso	Llegada	t
A	0	8
B	2	13
C	4	3
D	4	6
E	6	8
F	6	3

   Desarrolle la representación gráfica de cómo el despachador les asignaría el CPU, y la tabla de análisis, bajo:

   - Ronda con $q = 1$
   - Ronda con $q = 3$
   - Proceso más corto a continuación
   - Retroalimentación multinivel con $q = 1$, $n = 1$ y $Q = nq$

   Compare el rendimiento bajo cada uno de estos esquemas. ¿Qué ventajas presenta cada uno?

3. ¿Cuáles de los algoritmos estudiados son más susceptibles a la inanición que se presenta cuando $\rho > 1$? ¿Cuáles menos? Identifique por lo menos dos en cada caso.

4. Evalúe al planificador *por lotería* (sección 25.7).

   - ¿Cómo se compararía este método con los otros abordados?
   - ¿Para qué tipo de carga es más apto y menos apto?
   - ¿Qué tan susceptible resulta a producir inanición?
   - ¿Qué tan *justa* sería su ejecución?
   - ¿Qué modificaciones requeriría para planificar procesos con necesidades de tiempo real?

5. Tanto la afinidad a procesador como el balanceo de cargas son elementos importantes y deseables en todo planificador que se ejecute en un entorno multiprocesador. Sin embargo, afinidad y balanceo de cargas trabajan uno en contra del otro.

   Explique la anterior afirmación, y elabore cuándo debe predominar cada uno de estos mecanismos.

29.2. Temas de investigación sugeridos

Planificación en sistemas operativos *reales* A lo largo del presente capítulo se expusieron los principales esquemas base de planificación, si bien enunciados muy en líneas generales. Se mencionó también que muchos de estos esquemas pueden emplearse de forma híbrida, y que su comportamiento puede ser parametrizado llevando a resultados muy distintos.

Saliendo de la teoría hacia el terreno de lo aplicado, elija dos sistemas operativos en uso *relativamente común* en ambientes de producción, y averigüe qué mecanismos de planificación emplean.

¿A cuáles de los esquemas presentados se parece más? Los sistemas evaluados, ¿ofrecen al usuario o administrador alguna interfaz para cambiar el mecanismo empleado o ajustar sus parámetros? ¿Hay configuraciones distintas que puedan emplearse en este ámbito para sistemas de escritorio, embebidos, móviles o servidores?

Núcleo prevenible, tiempo real y optimización fina Los sistemas operativos modernos buscan *exprimir* hasta el último pedacito de rendimiento. Para estudiar cómo lo hacen, resulta interesante seguir las discusiones (y a la implementación) de Linux. Los últimos 10 años han sido de fuerte profesionalización y optimización.

Para el tema de planificación de procesos, un punto muy importante fue la introducción del *kernel prevenible* (o *interrumpible*), en 2004.

¿Qué significa que el núcleo mismo del sistema operativo puede ser interrumpido, quién lo puede interrumpir, qué consecuencias tuvo esto, en complejidad de código y en velocidad?

Pueden basarse para la preparación de este tema en un interesante reporte de NIST (Instituto Nacional de Estándares y Tecnología) de diciembre del 2002, Introduction to Linux for Real-Time Control: Introductory Guidelines and Reference for Control Engineers and Managers (Aeolean Inc. 2002). Detalla muy bien cuáles son los requisitos (y las razones detrás de ellos) para la implementación de sistemas operativos de tiempo real, e incluye una revisión del panorama de este campo, muy interesante y muy buen recurso para construir desde ahí.

Otra referencia muy recomendable al respecto es el artículo publicado por *Linux Weekly News*, Optimizing preemption (Corbet 2013c).

Si bien este tema toca principalmente temas de planificación de procesos, estos temas van relacionados muy de cerca con los presentados en el capítulo V y, particularmente, en las secciones 30.4 y 35.1.

Las clases de planificación en Linux y `SCHED_DEADLINE` En Linux, a partir de la versión 2.6.23, se usa un mecanismo de planificación llamado *planificador completamente justo* (*completely fair scheduler*). Sin embargo, para responder a procesos específicos con necesidades diferentes, Linux mantiene también *clases de planificación* independientes, las primeras de ellas más parecidas a los mecanismos simples aquí cubiertos: `SCHED_RR` y `SCHED_FIFO`.

En marzo del 2014, con el anuncio de la versión 3.14 del kernel, fue agregada la clase `SCHED_DEADLINE`, principalmente pensada para dar soporte a las necesidades de tiempo real. Esta opera bajo una lógica EDF (*primer plazo primero*, *earliest deadline first*).

Además de revisar este algoritmo, resulta interesante comprender cómo el sistema da soporte a la *mezcla* de distintas clases de planificación en un mismo sistema vivo; se sugiere hacer un breve análisis acerca de qué tanto `EDF` (o `SCHED_DEADLINE`, esta implementación específica) es tiempo real suave o duro.

Algunos textos que permiten profundizar al respecto:

- El artículo *Deadline scheduling: coming soon?* (Corbet 2013b), anticipando la inclusión de `SCHED_DEADLINE`, que presenta los conceptos, e incluye una discusión al respecto que puede resultarles interesante y útil.
- La página en *Wikipedia* de la clase de planificación `SCHED_DEADLINE` (Wikipedia 2011-2014).
- La documentación de `SCHED_DEADLINE` en el núcleo de Linux (Linux Project 2014).

29.3. Lecturas relacionadas

- P. A. Krishnan (1999-2009). *Simulation of CPU Process scheduling*. URL: `http://stimulationofcp.sourceforge.net/`; programa en Java desarrollado para un curso de sistemas operativos, presentando la simulación de distintos algoritmos de planificación.
- Neil Coffey (2013). *Thread Scheduling (ctd): quanta, switching and scheduling algorithms*. URL: `http://www.javamex.com/tutorials/threads/thread_scheduling_2.shtml`
- WikiBooks.org (2007-2013). *Microprocessor Design: Pipelined Processors*. URL: `http://en.wikibooks.org/wiki/Microprocessor_Design/Pipelined_Processors`
- Beth Plale (2003). *Thread scheduling and synchronization*. URL: `http://www.cs.indiana.edu/classes/b534-plal/ClassNotes/sched-synch-details4.pdf`
- *Páginas de manual* de Linux:
  - Linux man pages project (2014). *pthreads: POSIX threads*. URL: `http://man7.org/linux/man-pages/man7/pthreads.7.html`,
  - Linux man pages project (2008). *pthread_attr_setscope: set/get contention scope attribute in thread attributes object*. URL: `http://man7.org/linux/man-pages/man3/pthread_attr_setscope.3.html`,
  - Linux man pages project (2012). *sched_setscheduler: set and get scheduling policy/parameters*. URL: `http://man7.org/linux/man-pages/man2/sched_setscheduler.2.html`
- Mark Russinovich, David A. Solomona y Alex Ionescu (2012). *Windows Internals*. 6.ª ed. Microsoft Press. ISBN: 978-0735648739. URL: `http://technet.microsoft.com/en-us/sysinternals/bb963901.aspx`. El capítulo 5 aborda a profundidad los temas de hilos y procesos bajo Windows, y está disponible como ejemplo del libro para su descarga en la página referida.
- Jonathan Corbet (2013c). *Optimizing preemption*. URL: `https://lwn.net/Articles/563185/`

Parte V

Administración de memoria

Capítulo 30

Funciones y operaciones

El único espacio de almacenamiento que el procesador puede utilizar directamente, más allá de los registros (que si bien le son internos y sumamente rápidos, son de capacidad demasiado limitada) es la memoria física. Todas las arquitecturas de procesador tienen instrucciones para interactuar con la memoria, pero ninguna lo tiene para hacerlo con medios *persistentes* de almacenamiento, como las unidades de disco. Cabe mencionar que cuando se encuentre en un texto referencia al *almacenamiento primario* siempre se referirá a la memoria, mientras que el *almacenamiento secundario* se refiere a los discos u otros medios de almacenamiento persistente.

Todos los programas a ejecutar deben cargarse a la memoria del sistema antes de ser utilizados. En este capítulo se mostrará cómo el sistema operativo administra la memoria para permitir que varios procesos la compartan — esta tarea debe preverse desde el proceso de compilación de los programas (en particular, la fase de *ligado*). Hoy en día, además, casi todos los sistemas operativos emplean implementaciones que requieren de hardware especializado: la *Unidad de Manejo de Memoria* (MMU, por sus siglas en inglés). En el transcurso de este capítulo se describirá cómo se manejaban los sistemas multitarea antes de la universalización de las MMU, y qué papel juegan hoy en día.

En esta primer sección se presentarán algunos conceptos base que se emplearán en las secciones subsecuentes.

30.1. Espacio de direccionamiento

La memoria está estructurada como un arreglo direccionable de bytes. Esto es, al solicitar el contenido de una dirección específica de memoria, el hardware entregará un byte (8 bits), y no menos. Si se requiere hacer una operación sobre *bits* específicos, se deberá solicitar y almacenar bytes enteros. En algunas arquitecturas, el *tamaño de palabra* es mayor —por ejemplo, los procesadores Alpha incurrían en *fallas de alineación* si se solicitaba una dirección de memoria no alineada a 64 bits, y toda llamada a direcciones *mal alineadas* tenía que ser *atrapada* por el sistema operativo, re-alineada, y entregada.